



汕头大学数计学院计算机系

---

# 高级算法原理实践 项目报告

版本 **1.0**

组别：第一组

---

学 生：

黄应辉 (2024611031)

徐锦源 (2024611176)

罗展彬 (2024611093)

郑逸驰 (2024611021)

冯思语 (2024611197)

---

指导老师：

廖海泳、陈银冬

---

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

### 分工描述

| 姓名                  | 职责      | 主要工作                                                                             |
|---------------------|---------|----------------------------------------------------------------------------------|
| 黄应辉<br>(2024611031) | 组长      | 总体架构与进度把控；包骨架与 CI 三平台矩阵；模块接口契约定稿；集成联调与中期演示；三端打包发布与 GitHub Release；可行性研究报告与提交材料组装 |
| 徐锦源<br>(2024611176) | 算法内核开发者 | 关系文本解析与容错、不可变图结构、最长路径分层、Kahn 显式栈回溯全序枚举、环检测（全部自研）；概要设计主责                          |
| 罗展彬<br>(2024611093) | 事件引擎开发者 | 六类 StepEvent 协议、Kahn 步进内核、伪并发时间线（tick 交错/暂停/单步/调速/泳道上限）、golden 事件流；测试报告主责        |
| 郑逸驰<br>(2024611021) | 渲染场景开发者 | 分层画布、动画效果、候选池芯片、PNG 导出；渲染状态机与配色方案                                                |
| 冯思语<br>(2024611197) | 界面壳开发者  | 三栏主窗口、输入面板、控制条、泳道面板、结果流、两类错误提示；需求分析与用户手册主责，项目报告组装                                |

### 修订历史记录

| 日期         | 版本  | 说明                             | 作者                                      |
|------------|-----|--------------------------------|-----------------------------------------|
| 2026-09-15 | 0.1 | 技术栈定案，模块接口契约定稿，包骨架与 CI 三平台矩阵就位 | 黄应辉<br>(2024611031)                     |
| 2026-09-18 | 0.2 | 算法内核完成；修正标准图拓扑序基准（6 条修正为 7 条）  | 徐锦源<br>(2024611176)                     |
| 2026-09-19 | 0.3 | 事件引擎与渲染场景完成；概要设计、详细设计初稿就位      | 罗展彬<br>(2024611093)、郑逸驰<br>(2024611021) |
| 2026-09-20 | 0.4 | 界面壳完成；四模块集成联调打通；中期检查演示通过       | 冯思语<br>(2024611197)、黄应辉<br>(2024611031) |
| 2026-09-21 | 0.9 | 三端打包发布完成，产物发布至 Release v0.1.0  | 黄应辉<br>(2024611031)                     |
| 2026-09-23 | 1.0 | 交付文档补齐（可行性研究报告、测试报告）；提交材料组装    | 全组                                      |

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

目录

1. 软硬件环境.....3

2. 需求分析.....4

3. 详细设计.....5

    3.1 类设计.....5

    3.2 核心流程描述.....6

    3.3 核心算法设计.....7

4. 运行结果截图.....8

5. 测试（测试用例设计、运行结果）.....11

6. 系统特色以及可扩展点.....12

7. 感想.....13

8. 附件（会议记录）.....13

拓扑排序应用软件 项目报告

1.软硬件环境

开发环境以 Ubuntu 24.04.5 LTS (x86\_64) 与 macOS 为主机，语言为 Python 3.11.16，界面框架 PySide6 6.11.2 (Qt 6)，打包工具 PyInstaller 6.22.3，测试框架 pytest，依赖与工程管理统一使用 uv，依赖只从 pyproject.toml 声明、由 uv.lock 锁定版本，禁止裸 pip 安装。版本控制为 Git，持续集成为 GitHub Actions，矩阵覆盖 ubuntu-latest、macos-latest、windows-latest 三个平台。详见表 1-1。

表 1-1 开发环境

|         |                                   |
|---------|-----------------------------------|
| 项目      | 配置                                |
| 操作系统    | Ubuntu 24.04.5 LTS (x86_64)、macOS |
| 编程语言    | Python 3.11.16                    |
| GUI 框架  | PySide6 6.11.2 (Qt 6)             |
| 打包工具    | PyInstaller 6.22.3                |
| 测试框架    | pytest 9.1.1、pytest-qt 4.5.0      |
| 工程管理    | uv (pyproject.toml / uv.lock)     |
| 版本控制与集成 | Git、GitHub Actions 三平台矩阵          |

交付产物为已打包的可执行文件，目标机器无需安装 Python 或任何依赖库，三端各一份，均已发布并附同名校验文件，如表 1-2 所示。

表 1-2 三端交付产物

|             |                    |         |      |
|-------------|--------------------|---------|------|
| 平台          | 产物                 | 体积      | 打包模式 |
| Linux x64   | TopoSort           | 64.0 MB | 单文件  |
| Windows x64 | TopoSort.exe       | 46.5 MB | 单文件  |
| macOS       | TopoSort-macos.zip | 35.0 MB | 目录包  |

开发工具链约定三条：路径一律使用 pathlib 拼接，不硬编码分隔符；文件读写显式指定 UTF-8 编码，写文件使用空 newline 以保证往返一致；算法层与事件层导入后不得出现 PySide6 或 PyQt 模块，由集成环境机械检查强制。

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

## 2.需求分析

拓扑排序解决的是有先后约束的任务如何排出合法顺序的问题：学生选课要满足先修关系，汽车配件装配要满足工序依赖，复杂项目要满足任务依赖。约束较多时人工推排既慢又易漏解，而合法顺序往往不止一种，只看一条结果无法判断排布空间。本项目据此实现一个桌面应用：输入先后关系，画出关系图，用动画演示排序全过程，给出尽可能多的拓扑排序结果，并在约束自相矛盾时识别出来。目标用户为学习数据结构与算法的学生、需要梳理先后约束的教学或工程人员，典型场景包括课堂演示、作业校验、冲突排查与结果留档。功能需求共 22 条，按输入、图形展示、动画演示、结果输出、异常边界五组归类，标注 P0 必须实现与 P1 功能扩充两级优先级，同类项合并后的功能需求如表 2-1 所示。

表 2-1 功能需求

| 编号       | 功能需求                                    | 优先级   |
|----------|-----------------------------------------|-------|
| FR-01/02 | 界面手工输入关系，一行一条尖括号格式；从 UTF-8 文本文件导入       | P0    |
| FR-03/04 | 空白行跳过，全角括号与逗号、省略尖括号均可接受；重复关系去重          | P1/P0 |
| FR-05/06 | 非法输入报错并指明行号；关系可保存为文本文件且往返一致             | P0/P1 |
| FR-07/08 | 自动绘制有向关系图并按依赖层次分层；画布支持平移与缩放             | P0/P1 |
| FR-09/10 | 节点区分待处理、就绪、本拍选中、已消耗、卡住五种视觉状态；候选池醒目呈现    | P0    |
| FR-11    | 开始后动画演示排序全过程，节点入池、被选中消耗、图上同步变化          | P0    |
| FR-12    | 候选池有多个可选节点时多路并推，界面以泳道并列展示各分支            | P0    |
| FR-13/14 | 支持暂停、恢复、单步、跳完与调速；演示可复现，两次运行步骤序列一致       | P0    |
| FR-15    | 分支数超过泳道展示上限时超出部分不单独展示但仍计入结果并提示          | P1    |
| FR-16/17 | 列出尽可能多的合法拓扑序，每条一行空格分隔；超限截断并显示实际条数，界面不卡死 | P0    |
| FR-18/19 | 结果导出为文本文件；支持仅计数模式用于大图快速估算               | P1    |
| FR-20    | 检出环时明确提示并列出卡住节点，说明不存在拓扑序且不崩溃            | P0    |
| FR-21/22 | 边界输入覆盖空输入、单节点、无边、全链、稠密分叉、多环、自环；自环视为非法   | P0    |

非功能需求共 7 条，其中三端通用、算法原创、分层可维护为课程与项目的硬性约束，见表 2-2。

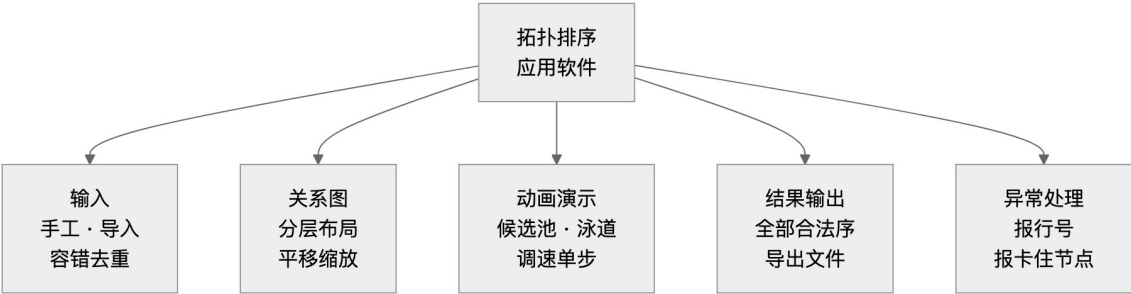
表 2-2 非功能需求

| 编号     | 非功能需求                                 | 验收方式   |
|--------|---------------------------------------|--------|
| NFR-01 | 三端通用，不使用平台专属接口，路径用 pathlib，读写显式 UTF-8 | 三平台矩阵  |
| NFR-02 | Kahn 排序、全序枚举、环检测全部自研，不用现成排序库          | 课程考核点  |
| NFR-03 | 30 至 60 节点稀疏图在极速模式下数秒内给出结果或计数         | 性能实测记录 |
| NFR-04 | 分层架构，算法层零 GUI 依赖且可独立测试，模块依赖单向         | 机械断言   |
| NFR-05 | 窗口默认 1280×800 三栏布局，动画流畅，错误提示可见且指出原因   | 界面手册   |
| NFR-06 | 单文件可执行程序三端各一份，目标机无需 Python 环境         | 打包产物实测 |
| NFR-07 | 决策留档、可见功能配截图、测试输入入库，保证可复现             | 项目规范   |

输入输出统一为纯文本：输入一行一条关系，输出一行一条完整拓扑序、空格分隔。教材标准图由四条边构成，其全部合法拓扑序恰为 7 条，该数量口径经穷举复核后自 2026-09-19 起由原 6 条修正为 7 条，修正依据存于决策档案。系统功能构成如图 2-1 所示。验收标准有五条：用课程先修关

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

系示例测试运算结果正确；用计算机系学业指南整理的真实课程先修关系作图并导出排序结果；满足全部 P0 需求且 P1 实现情况记入测试报告；考虑非法输入、环、超大规模、空输入等突发情况且不崩溃；三端各有一份可运行产物。



3.详细设计

系统按界面层、渲染层、事件层、算法层四层单向依赖组织，依赖方向为界面层指向渲染层与事件层与算法层、渲染层指向事件层、事件层指向算法层，禁止反向与跨层依赖，算法层与事件层两层零Qt 依赖。随后依次给出类设计、核心流程描述与核心算法设计。

3.1类设计

系统四层架构与依赖方向如图 3-1 所示。



各层职责与关键类型见表 3-1。

表 3-1 模块与关键类型

| 模块  | 关键类型或函数             | 职责                            |
|-----|---------------------|-------------------------------|
| 算法层 | ParseError、parse()  | 文本解析、去重边、拒绝自环，错误携带行号与原文       |
| 算法层 | Graph、CycleError    | 不可变图结构（frozenset 节点集与边集）与公共查询 |
| 算法层 | Kahn 分析、显式栈搜索       | 入度与最长路径层号计算，稳定枚举或仅计数全部拓扑序     |
| 事件层 | 六类事件数据类             | 入池、消费、分叉、完成、死路、环六类事件的唯一定义处    |
| 事件层 | TopoPlayer、Timeline | 步进内核产出确定性事件流；伪并发调度与泳道上限       |

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

|     |                                  |                            |
|-----|----------------------------------|----------------------------|
| 渲染层 | GraphBoard、Effects、CandidatePool | 分层图元布局、动画时长集中管理、候选池芯片与导出   |
| 界面层 | MainWindow 等五个文件                 | 组装与信号槽接线，输入、控制、结果呈现与两类错误提示 |

不可变性是算法层的关键设计：图的节点集与边集使用 `frozenset`，算法运行时只使用局部邻接表、入度表与栈帧，不修改图对象，因此同一个图可被重复播放并得到确定结果。界面层对外只暴露主窗口的七个公共方法，覆盖设置与读取输入、启动全链路、读取结果、读取错误文案、文件存取往返，测试全部经由这些方法驱动，不模拟点击真实按钮。

3.2核心流程描述

一次完整运行的数据流为：用户输入文本经解析得到图对象；图对象一方面经分层计算得到画布静态布局，另一方面交给播放器与时间线，按拍产出事件列表；渲染层消费事件完成节点状态迁移与动画，界面不自行计算排序，只消费事件流；完成事件累积为结果列表并支持导出；解析失败或检出环时进入错误提示分支。处理流程如图 3-2 所示。

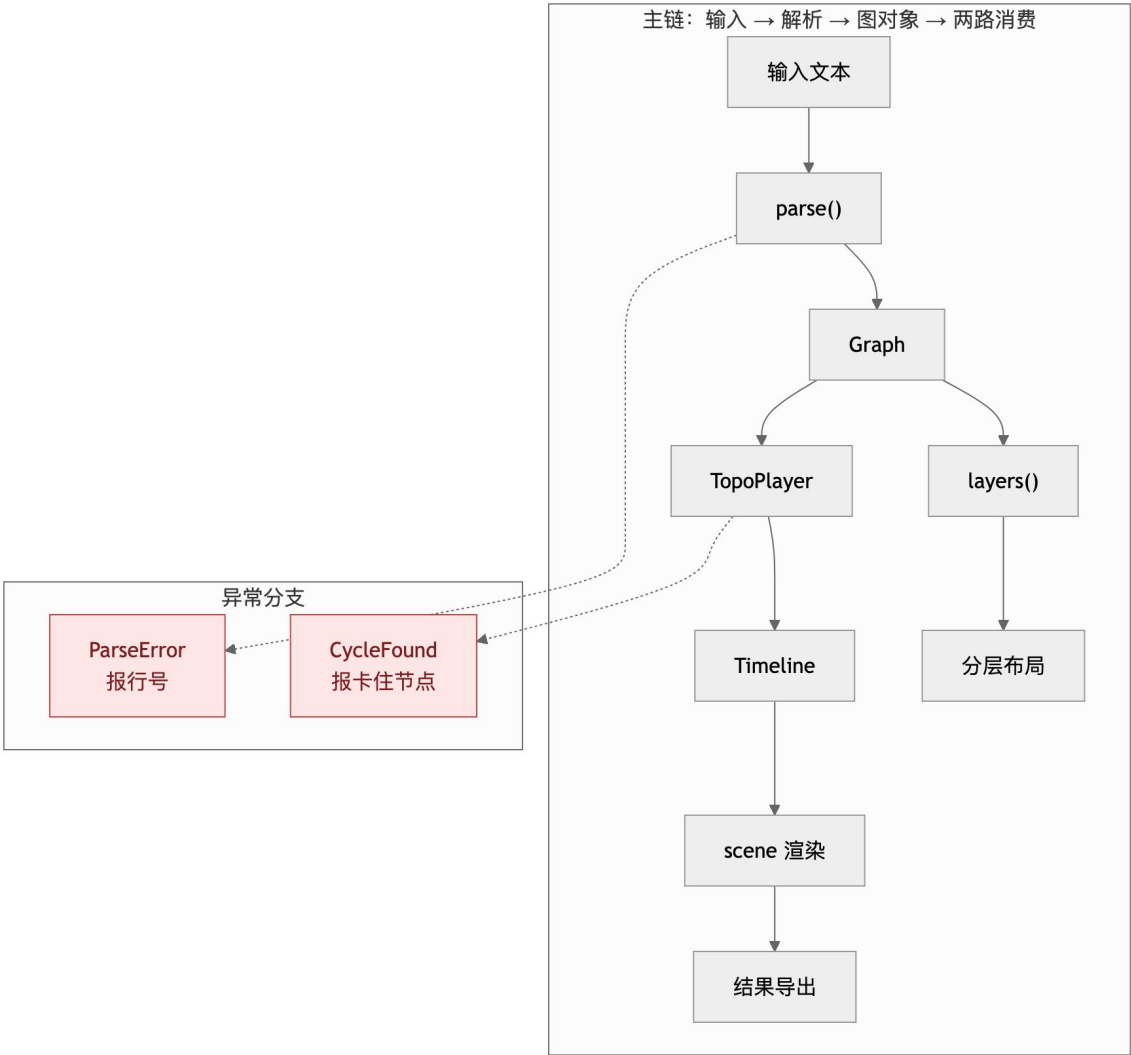


图 3-2 系统处理流程

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

节点状态机为：待命节点经入池事件转为就绪，经消费事件转为本拍选中并播放脉冲动画，随即稳定为已被消费并保留原位半透明以留存过程轨迹；环及其下游节点为异常态、红色显示。对外只暴露待命、就绪、已消费、卡住四种稳定状态，本拍选中是瞬时视觉样式而非可查询状态，测试因此断言稳定状态而不依赖动画中间帧。节点状态迁移如图 3-3 所示。

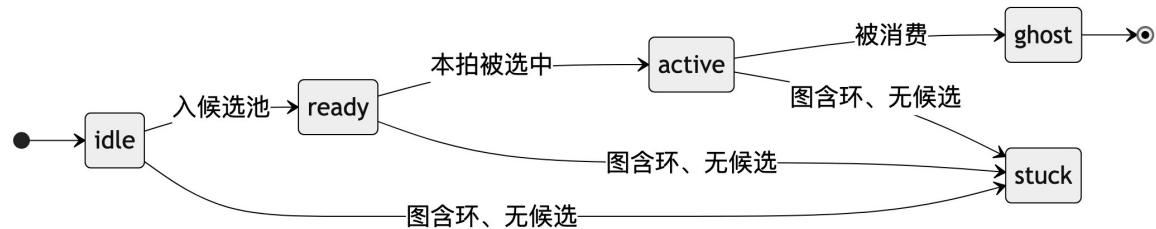


图 3-3 节点视觉状态机

3.3核心算法设计

本项目的三个算法全部自研，不使用现成图论库的排序结果。Kahn 分析在预处理阶段由边集构造按节点名排序的邻接表并计算入度，零入度节点构成初始候选集；处理节点时对每条出边更新后继节点的层号为当前层号加一与原值的较大者，并令其后继入度减一，入度降为零时该节点可继续处理。扫描结束后若所有节点均已释放，则图无环，层号即从源点出发的最长路径长度，直接用于固定布局，因此无需引入自动布局库；若仍有节点未释放，则剩余节点入度全部不小于一，构成卡住节点集，分层接口遇到该情况抛出环异常而不返回不完整布局。单次分析的时间与空间复杂度均为节点数与边数之和的量级。

全序枚举把候选选择动作扩展为搜索树：某一步候选集有 k 个节点时，原分支按字典序选最小者消费，其余 k 减一个备选各开一条新分支；新分支首次推进必须消费分叉时绑定的备选节点而不重新自选，否则各分支会收敛到同一条序。为避免递归深度限制，搜索使用显式栈，栈帧保存当前候选元组、下一个候选下标与进入该层时选择的节点，分支结束时弹出栈帧并恢复后继入度与路径。按字典序深度优先展开保证输出顺序稳定，每条分支恰好对应一条合法序，不重复也不遗漏，分支展开结构如图 3-4 所示。仅计数模式复用同一状态机，叶节点只将计数器加一，不复制与保存完整序列，降低结果存储开销，但仍需访问搜索树的每个叶子。

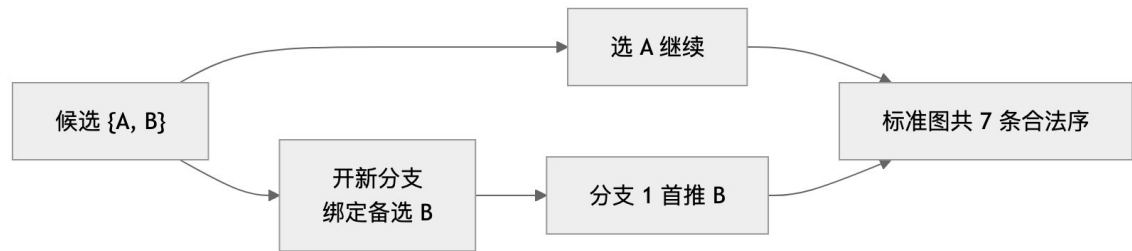


图 3-4 全序枚举的分支展开

环检测是 Kahn 的副产品，语义为环与环下游节点的并集：某分支无候选但仍有剩余节点即进入死路，全部分支终止后若有分支卡死则发出环事件，取各分支卡住节点的并集。该语义的等价性在于候选集为空恰好意味着剩余节点入度全部不小于一，所以卡住的不只是环上节点，还包括被环阻塞的下游节点。标准图枚举正确性经独立复核：实现输出 7 条序与既有基准的 6 条冲突，通过穷举全部 120 个五元排列确认原清单遗漏了合法序，算法本身正确，最终修正基准为 7 条，而不是让算法迁就错误计数。

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

确定性保证是一项隐蔽要求：动画演示要可复现，同一输入的事件序列必须逐项相等，而集合的迭代顺序受哈希种子影响，跨进程与跨平台并不固定。对策写入契约并由集成环境强制：一切集合遍历与事件发射顺序一律按字典序排序，序列使用元组、节点集使用 `frozenset` 以保证可哈希、可比与内容序稳定，该性质由固化的期望事件流文件逐项比对锁定。

规模防护针对阶乘级爆炸设置两级措施：（1）截断参数，累计完成数达上限即停止推进，已得结果保留；（2）前置保险丝，仅按完成数点火太晚，改为开新分支前检查已完成数与活跃分支数之和是否达到上限，达到即不再分叉，在爆炸发生前掐断。该设计有实测触发背景：逐拍交错调度下首条分支需若干步才产出首个完成事件，宽并行图此前分支已按组合数爆炸，30 节点双链曾出现 26 秒零产出。界面层另设结果条数上限 2000，由算法层截断，防止稠密分叉图卡死界面。需要说明的是，精确计数等价于统计偏序的线性扩展个数，一般情形属于计数复杂度类难题，极速模式只表示避免构造结果列表，真正决定耗时的是搜索树叶节点总数而非节点数本身。

4.运行结果截图

输入教材标准图的四条关系后，画布按最长路径分层绘制关系图，节点按依赖层次排布，源点位于第零层，如图 4-1 所示。

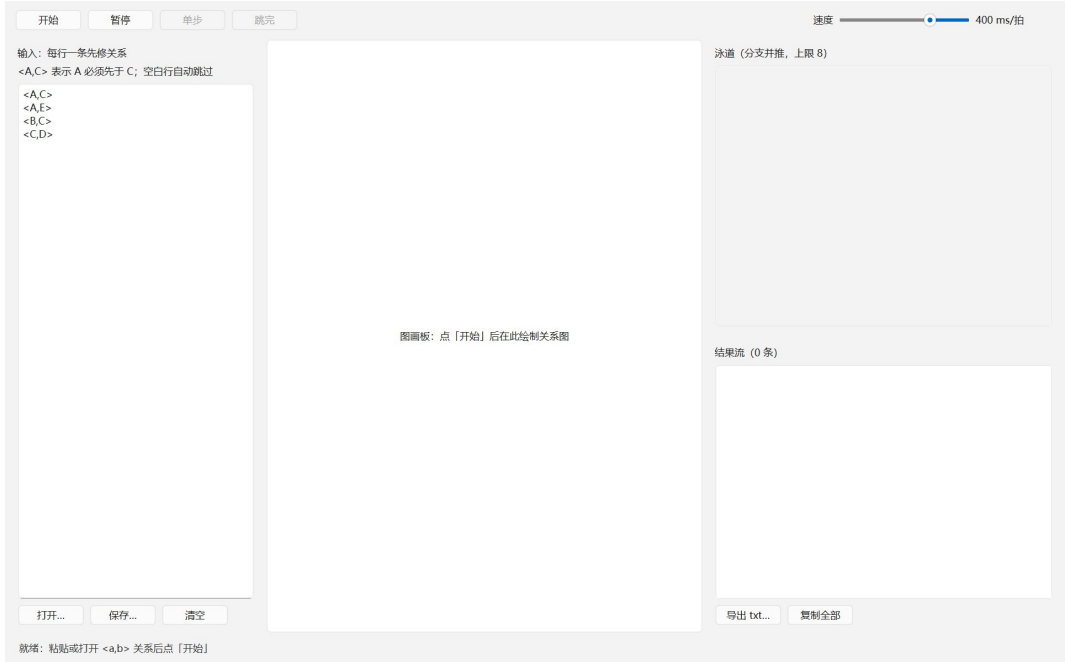


图 4-1 输入完成后的主窗口与分层关系图

演示过程中候选池出现多个可选节点时，程序分出多条分支并在泳道区并列展示，直观呈现同一时刻存在多种合法选择，如图 4-2 所示。

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

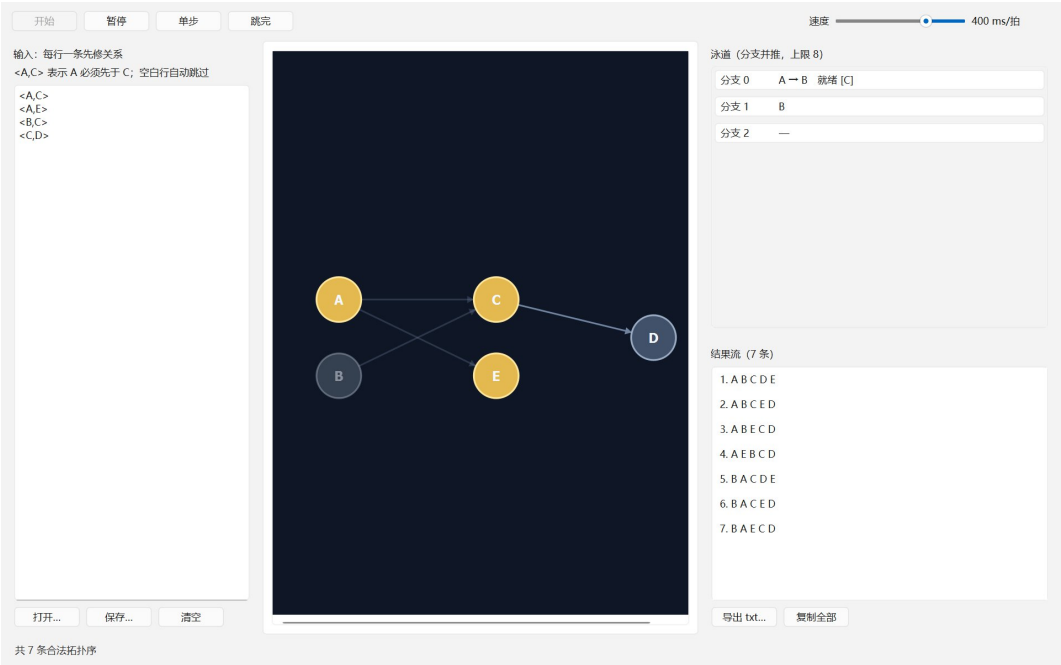


图 4-2 分叉多路伪并发泳道

运行结束后结果流列出标准图的全部 7 条合法拓扑序，每条一行、空格分隔，如图 4-3 所示。

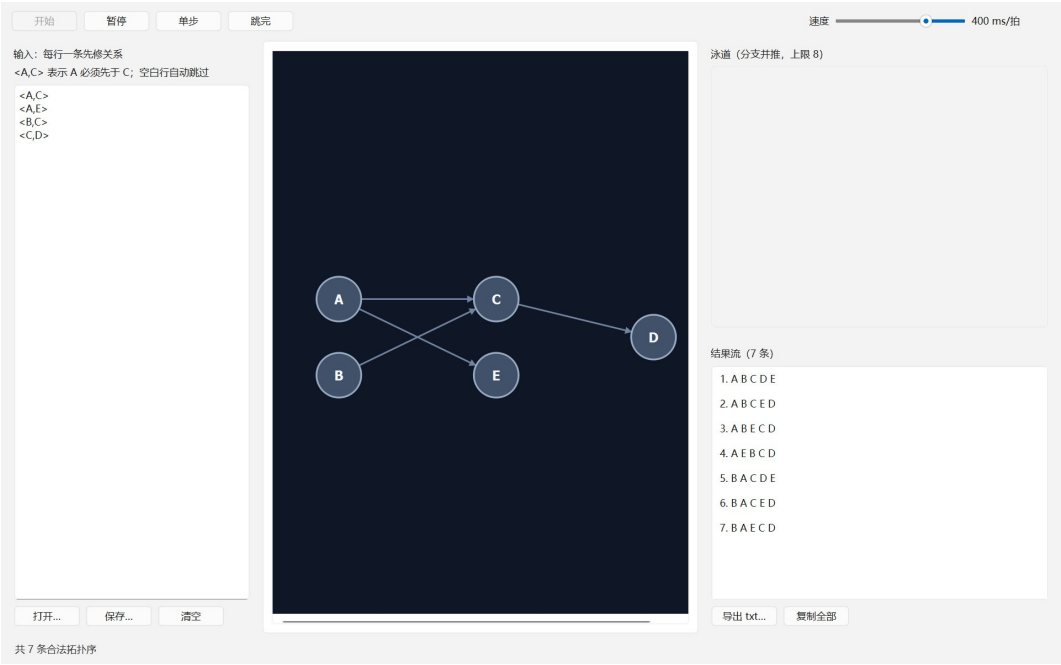


图 4-3 结果流列出 7 条合法拓扑序

输入互锁的两条关系后，状态行红底提示检测到环并列出卡住节点，说明不存在拓扑序，如图 4-4 所示。

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

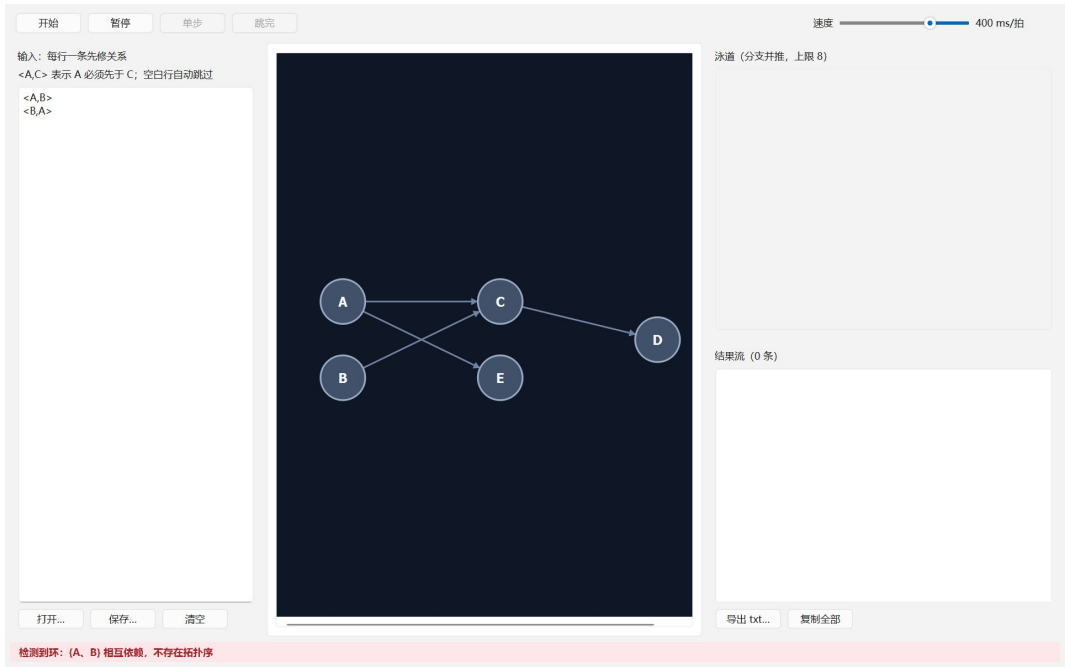


图 4-4 环检测提示

缺逗号等非法输入触发解析错误提示，直接指明出错行号，便于回查原始输入，如图 4-5 所示。

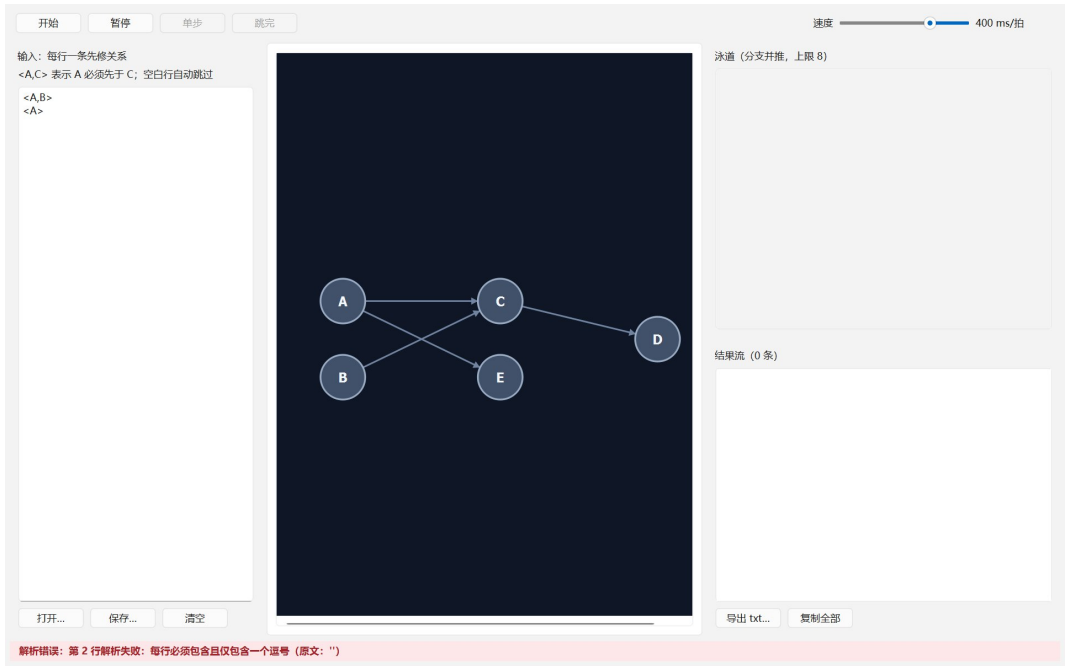


图 4-5 非法输入提示（带行号）

交付的单文件产物在真实桌面环境运行，非开发环境且无需任何 Python 依赖，如图 4-6 所示。

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |



图 4-6 打包产物运行

5.测试（测试用例设计、运行结果）

测试策略由四部分构成：开发前先写契约测试作为路标，每条红测试对应一个未完成模块；以机械红线强制算法层与事件层零 Qt 依赖、并禁止用字符串拼接路径；用固化的期望事件流锁定确定性；测试输入一律从素材库读取而不硬编码。持续集成分两层判定：契约红线必须全绿并受分支保护，模块测试的红只作为完成度路标而不阻塞他人。用例库共 25 个文件、11 组用例，覆盖正常路径、环与边界、坏输入、大图四类，全部要求报行号且不崩溃，分组设计见表 5-1。

表 5-1 测试用例分组设计

| 类型    | 用例                | 预期结果             |
|-------|-------------------|------------------|
| 正常路径  | 标准五节点（教材示例）       | 恰 7 条合法序，无环      |
| 容错解析  | 宽容格式与重复边          | 去重后恰 1 条序        |
| 环与边界  | 二节点环、环带下游、环与下游阻塞  | 序为空，卡住集合环上与下游节点  |
| 坏输入   | 缺逗号、自环、多余括号       | 均报行号且不崩溃         |
| 大图    | 30 节点稀疏链、60 节点稀疏链 | 结果唯一，用于性能测量      |
| 代码级补充 | 空图、空图画布导出         | 空图恰发一次完成事件并可导出图片 |

在本机执行全部测试的真实输出为 66 个用例全部通过，耗时 0.22 秒；在另一台机器复跑同一命令同样 66 个用例全部通过，耗时差异来自两个系统无显示环境初始化等开销。分模块统计见表 5-2。

表 5-2 分模块测试结果

| 测试文件 | 用例数 | 结果 | 覆盖内容                          |
|------|-----|----|-------------------------------|
| 契约测试 | 2   | 通过 | 零 Qt 红线、路径拼接红线                |
| 算法内核 | 15  | 通过 | 解析、去重、容错、报行号、环检测、7 条序、截断、分层   |
| 事件引擎 | 25  | 通过 | 确定性、期望事件流比对、泳道上限、状态机、单步、分支语义  |
| 渲染场景 | 20  | 通过 | 画布状态迁移、边淡化、平移缩放不改语义、导出、动画时长有界 |
| 界面壳  | 4   | 通过 | 端到端冒烟、两类错误提示、文件存取往返           |

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

性能按极速模式实测并逐行入库：30 节点稀疏链为 30 节点 29 边、结果数 1、耗时 0.061 毫秒；60 节点稀疏链为 60 节点 59 边、结果数 1、耗时 0.083 毫秒。节点数翻倍耗时仅增约三成，与节点数加边数的线性预期吻合，远优于数秒内的指标要求。需要说明的是全链是计数的最简情形，真正决定耗时的是搜索树叶子总数，分叉多的图计数会随分支指数增长。性能与三端打包实测汇总见表 5-3。

表 5-3 性能与打包实测

| 用例            | 规模         | 结果数 | 耗时               |
|---------------|------------|-----|------------------|
| 30 节点稀疏链（仅计数） | 30 节点 29 边 | 1   | 0.061 ms         |
| 60 节点稀疏链（仅计数） | 60 节点 59 边 | 1   | 0.083 ms         |
| Linux 发布包     | 单文件、strip  | —   | 38.8 s / 64.0 MB |
| Windows 发布包   | 单文件        | —   | 68.2 s / 46.5 MB |
| macOS 发布包     | 目录包        | —   | 26.5 s / 35.0 MB |

表 5-4 缺陷记录

| 缺陷               | 根因                     | 修复           |
|------------------|------------------------|--------------|
| Windows 构建打印中文即崩 | 控制台默认非 UTF-8 触发编码异常    | 输出统一改为 UTF-8 |
| Windows 冒烟误判失败   | 被终止进程短暂持有句柄，临时目录清理抛异常  | 清理容错并留终止宽限   |
| 三平台构建全绿但发布失败     | 发布任务无版本库目录，命令行工具无法推断仓库 | 显式注入仓库环境变量   |
| macOS 产物体积虚高     | 目录遍历未排除符号链接，同一实体被重复计入  | 统计时跳过符号链接    |
| 集成层任务整片变红        | 缺少系统图形库，本地与集成结论矛盾      | 补齐依赖库后全部转绿   |

开发过程中暴露并修复了 5 个真实缺陷，全部有持续集成运行号或决策档案佐证，摘要见表 5-4；其中三项只能由三平台持续集成发现，反向验证了本地只验单一系统会漏检的共性问题。测试结论：功能上 66 个自动化用例全部通过，覆盖正常路径、含下游连坐的环检测、三类环输入、容错解析、大图规模、动画确定性与界面端到端，用例可逐条回溯到需求编号；正确性上标准图的 7 条合法序经枚举、固化文件、单元测试与界面结果流四方一致；三端通用上契约红线在三个平台强制通过，三份产物均经回归测试、空目录启动冒烟与校验和自检后发布；性能上实测远优于指标要求，结果爆炸场景由截断上限保护界面不卡死。遗留事项如实披露：Windows 与 macOS 实机手工全流程验证尚未进行，目前证据为集成环境冒烟与静态结构校验；macOS 未做正式签名与公证，首次打开需解除隔离属性。界面呈现见图 4-3 与图 4-4。

6.系统特色以及可扩展点

系统特色可归纳为七点：（1）算法全自研且正确性经独立复核，枚举结果通过穷举全部排列验证，并据此修正了错误的基础数据，算法没有为迁就错误计数而漏解；（2）把算法过程变成看得见的多路并发，不止展示一条排序路径，而是用泳道伪并发同时呈现多条合法序的推进过程；（3）确定性可验收，通过集合遍历一律按字典序的契约级约束与固化事件流文件，把动画可复现从主观感受变成可机械比对的性质；（4）环检测语义完整，不只报告环上节点，还报告被环阻塞的下游节点，更符合用户排查依赖矛盾的实际需要；（5）规模风险有前置防护，截断参数与开分支前置保险丝在爆炸发生前掐断而非等超时；（6）三端一致交付，组内没有 Windows 开发机，通过三平台矩阵把这一约束转化为每次提交自动验证三端的优势，产物附校验值；（7）工程规范可追溯，十余篇决策档案与只增不改的素材库保证从零可复现，算法红线由机械检查强制。在上述基础上可扩展的方向如表 6-1 所示。

|               |            |
|---------------|------------|
| 拓扑排序应用软件 项目报告 | 版本：1.0     |
|               | 2026-09-26 |
| 第一组 · Group01 |            |

表 6-1 可扩展方向

| 方向        | 说明                       |
|-----------|--------------------------|
| 关键路径分析    | 在现有分层结果上直接计算最长路径，给出最短工期  |
| 逆拓扑序与反向演示 | 复用现有内核，增加反向步进即可          |
| 导出格式扩展    | 现有文本与图片，可增加数据交换格式与图描述语言  |
| 布局算法      | 当前为固定分层，可增加力导向或正交布局切换    |
| 增量输入      | 在已有图上增删边并局部重算，避免全量重建     |
| 性能优化      | 枚举层按分支并行化                |
| 国际化       | 界面文案抽取为资源文件，支持英文         |
| 体积优化      | 插件裁剪需自定义打包钩子并三端验证，已评估转待办 |

## 7.感想

项目由五名成员分工完成，各人在承担模块中的收获集中体现为对可复现、契约与确定性的重新认识，摘要见表 7-1，完整原文见各自的个人任务及感想文档。

表 7-1 成员分工与感想摘要

| 成员                  | 角色与主责           | 感想摘要                                            |
|---------------------|-----------------|-------------------------------------------------|
| 黄应辉<br>(2024611031) | 组长，基建契约、集成、打包发布 | 可复现比写得快更重要，决策留档让并行开发分歧显著减少，踩坑只有写成机械判定才算真正通用     |
| 徐锦源<br>(2024611176) | 算法内核开发者         | 把能跑的排序放进可交付软件，真正要处理的是边界、接口与可复现性，底层接口越清楚联调成本越低   |
| 罗展彬<br>(2024611093) | 事件引擎开发者         | 最大收获是确定性思维与契约先行，演示软件的标准是任意平台重跑逐帧一致，数错基准也要诚实修正   |
| 郑逸驰<br>(2024611021) | 渲染场景开发者         | 让渲染层只消费稳定事件与布局数据、在不同事件顺序下保持可预测的视觉状态，是分层解耦最具体的实践 |
| 冯思语<br>(2024611197) | 界面壳开发者，需求与手册文档  | 契约是协作的锚，因为错误的原因变绿比红着更危险，留痕不是负担而是写报告时的素材         |

## 8.附件（会议记录）

项目开发期间共召开 5 次小组会议，每次由不同成员担任记录人员，会议涉及的决策均可回溯到决策档案，会议记录见表 8-1。

表 8-1 小组会议记录

| 日期         | 主题      | 记录人员                | 要点                               |
|------------|---------|---------------------|----------------------------------|
| 2026-09-14 | 分组与任务分工 | 黄应辉<br>(2024611031) | 任务宣讲、五人模块分工、留档纪律与算法自研红线          |
| 2026-09-15 | 开工定案    | 冯思语<br>(2024611197) | 技术栈定案、三端通用要求、任务拆分与接口契约、骨架合入      |
| 2026-09-20 | 开发期进度同步 | 罗展彬<br>(2024611093) | 四模块并行开发、标准图 6 改 7 修正、事件引擎与渲染设计定案 |
| 2026-09-21 | 中期检查    | 徐锦源<br>(2024611176) | 集成合入、集成环境缺库定位、打包发布方案启动           |
| 2026-09-24 | 交付收尾    | 郑逸驰<br>(2024611021) | 三端产物发布、六份文档齐稿、个人章节与交付包分工         |